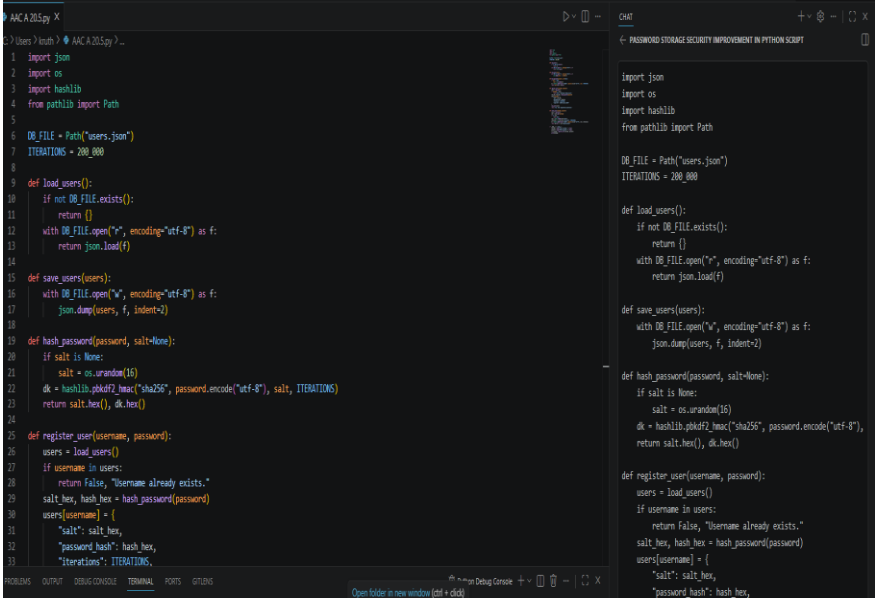


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week10 – Friday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number:20.5(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code	Week10 - Friday																		

	Lab Objectives: - Understand how to test AI-generated code for common security vulnerabilities. - Learn to apply secure coding principles while analyzing AI outputs. - Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption. - Enhance code reliability and safety by using AI for secure refactoring.	
	Task 1 – Password Storage Vulnerability Check Task: Analyze an AI-generated signup script for insecure password storage. Instructions: - Prompt AI to generate a Python user registration program. - Check whether passwords are stored in plain text. - Improve the script using hashing (bcrypt or hashlib). Expected Output: - A secure registration script with hashed password storage.  <pre> 1 import json 2 import os 3 import hashlib 4 from pathlib import Path 5 6 DB_FILE = Path("users.json") 7 ITERATIONS = 200_000 8 9 def load_users(): 10 if not DB_FILE.exists(): 11 return [] 12 with DB_FILE.open("r", encoding="utf-8") as f: 13 return json.load(f) 14 15 def save_users(users): 16 with DB_FILE.open("w", encoding="utf-8") as f: 17 json.dump(users, f, indent=2) 18 19 def hash_password(password, salt=None): 20 if salt is None: 21 salt = os.urandom(16) 22 dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS) 23 return salt.hex(), dk.hex() 24 25 def register_user(username, password): 26 users = load_users() 27 if username in users: 28 return False, "Username already exists." 29 salt_hex, hash_hex = hash_password(password) 30 users[username] = { 31 "salt": salt_hex, 32 "password_hash": hash_hex, 33 "iterations": ITERATIONS, 34 } 35 save_users(users) 36 return True, "User registered successfully." </pre> <pre> import json import os import hashlib from pathlib import Path DB_FILE = Path("users.json") ITERATIONS = 200_000 def load_users(): if not DB_FILE.exists(): return [] with DB_FILE.open("r", encoding="utf-8") as f: return json.load(f) def save_users(users): with DB_FILE.open("w", encoding="utf-8") as f: json.dump(users, f, indent=2) def hash_password(password, salt=None): if salt is None: salt = os.urandom(16) dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS) return salt_hex(), dk_hex() def register_user(username, password): users = load_users() if username in users: return False, "Username already exists." salt_hex, hash_hex = hash_password(password) users[username] = { "salt": salt_hex, "password_hash": hash_hex, "iterations": ITERATIONS, } save_users(users) return True, "User registered successfully." </pre>	

```

19 def hash_password(password, salt=None):
20     return salt.hex(), dk.hex()
21
22
23
24
25 def register_user(username, password):
26     users = load_users()
27     if username in users:
28         return False, "Username already exists."
29     salt_hex, hash_hex = hash_password(password)
30     users[username] = {
31         "salt": salt_hex,
32         "password_hash": hash_hex,
33         "iterations": ITERATIONS,
34         "algorithm": "pbkdf2_hmac_sha256"
35     }
36     save_users(users)
37     return True, "User registered successfully."
38
39
40 def verify_user(username, password):
41     users = load_users()
42     user = users.get(username)
43     if not user:
44         return False
45     salt = bytes.fromhex(user["salt"])
46     iterations = int(user.get("iterations", ITERATIONS))
47     dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)
48     return dk.hex() == user["password_hash"]
49
50
51 if __name__ == "__main__":
52     username = input("Enter username: ").strip()
53     password = input("Enter password: ").strip()
54     ok, message = register_user(username, password)
55     print(message)

```

```

import json
import os
import hashlib
from pathlib import Path

DB_FILE = Path("users.json")
ITERATIONS = 200_000

def load_users():
    if not DB_FILE.exists():
        return {}
    with DB_FILE.open("r", encoding="utf-8") as f:
        return json.load(f)

def save_users(users):
    with DB_FILE.open("w", encoding="utf-8") as f:
        json.dump(users, f, indent=2)

def hash_password(password, salt=None):
    if salt is None:
        salt = os.urandom(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations)
    return salt.hex(), dk.hex()

def register_user(username, password):
    users = load_users()
    if username in users:
        return False, "Username already exists."
    salt_hex, hash_hex = hash_password(password)
    users[username] = {

```

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

```

1 import os
2 import json
3 from pathlib import Path
4 import requests
5
6 BASE_URL = os.getenv("WEATHER_API_BASE_URL", "https://api.openweathermap.org/data/2.5/weather")
7 CONFIG_PATH = Path("config.json") with open("weather_config.json")
8
9 def load_api_key() -> str | None:
10     env_key = os.getenv("WEATHER_API_KEY")
11     if env_key:
12         return env_key
13     if CONFIG_PATH.exists():
14         with CONFIG_PATH.open("r", encoding="utf-8") as f:
15             data = json.load(f)
16             key = data.get("WEATHER_API_KEY")
17             if key:
18                 return key
19     return None
20
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33

```

```

1. Go to https://home.openweathermap.org/users/sign_up
2. Create an account and verify email
3. Open "API keys" page https://home.openweathermap.org/api_keys
4. Copy the generated key
5. Use it in your script when prompted, or set
6. PowerShell (current session): $env:WEATHER_API_KEY="your_key"
7. Or put it in: H:weather_config.json as {"WEATHER_API_KEY":"your_key"}

If you want, I can also switch your script to a weather API that has easier free-tier setup.

City: Pskov
Error: WEATHER_API_KEY: 8481c629579ae545a05310aa420b
Traceback (most recent call last):
  File "C:\Users\paul\AKA-A-2025.py", line 34, in <module>
    data = get_weather(city)
           ^^^^^^^^^^^^^
  File "C:\Users\paul\AKA-A-2025.py", line 25, in get_weather
    response.raise_for_status()
  File "C:\Users\paul\AppData\Local\Programs\Python\Python312\python.exe", line 100, in raise_for_status
    raise HTTPError(http_error_msg, response=self)
requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Pskov&appid=8481c629579ae545a05310aa420b&units=metric

You're hitting a real API authentication issue (401) so I'll harden AKA-A-2025.py to handle invalid keys gracefully and print a clear message instead of a traceback.

```

```
◆ AAC A 20.5.py X
C:\Users> kruth > ◆ AAC A 20.5.py > get_weather
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```
◆ AAC A 20.5.py X
C:\Users> kruth > ◆ AAC A 20.5.py >
1 from flask import Flask, request, jsonify
2 from werkzeug.utils import secure_filename
3 from pathlib import Path
4 import os
5
6 app = Flask(__name__)
7 app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
8
9 UPLOAD_DIR = Path("uploads")
10 UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
11
12 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf", "txt")
13
14 EXTENSION_TO_KIND = {
15     "png": ("img"),
16     "jpg": ("img"),
17     "jpeg": ("img"),
18     "pdf": ("pdf"),
19     "txt": ("text"),
20 }
21
22 def detect_kind(data: bytes) -> str:
23     if data.startswith(b"PNG") or data.startswith(b"JPG") or data.startswith(b"JPEG") or data.startswith(b"PDF") or data.startswith(b"TXT"):
24         return "img"
25     if data.startswith(b"PDF"):
26         return "pdf"
27     if data.startswith(b"TEXT"):
28         return "text"
29     if data.startswith(b"UNKNOWN"):
30         return "unknown"
31     if b"UNKNOWN" in data:
32         return "unknown"
33     return "unknown"
```

```
AAC A 20.5.py 2 X
C:\Users> kruth > AAC A 20.5.py > ...
21 def detect_kind(data: bytes) -> str:
34     try:
35         data.decode("utf-8")
36         return "txt"
37     except UnicodeDecodeError:
38         return "unknown"
39
40 @app.post("/upload")
41 def upload_file():
42     if "file" not in request.files:
43         return jsonify({"error": "No file part"}), 400
44
45     file = request.files["file"]
46     if file.filename is None or file.filename.strip() == "":
47         return jsonify({"error": "No selected file"}), 400
48
49     original_name = secure_filename(file.filename)
50     if original_name == "":
51         return jsonify({"error": "Invalid filename"}), 400
52
53     ext = Path(original_name).suffix.lower().lstrip(".")
54     if ext not in ALLOWED_EXTENSIONS:
55         return jsonify({"error": "File extension not allowed"}), 400
56
57     head = file.stream.read(8192)
58     if not head:
59         return jsonify({"error": "Empty file"}), 400
60
61     kind = detect_kind(head)
62     if kind in {"exe", "zip", "binary", "unknown"}:
63         return jsonify({"error": "Unsafe or unsupported file content"}), 400
64
65     if kind not in EXTENSION_TO_KIND[ext]:
```

```
AAC A 20.5.py 2 X
C:\Users> kruth > AAC A 20.5.py > ...
41 def upload_file():
50     if original_name == "":
51         return jsonify({"error": "Invalid filename"}), 400
52
53     ext = Path(original_name).suffix.lower().lstrip(".")
54     if ext not in ALLOWED_EXTENSIONS:
55         return jsonify({"error": "File extension not allowed"}), 400
56
57     head = file.stream.read(8192)
58     if not head:
59         return jsonify({"error": "Empty file"}), 400
60
61     kind = detect_kind(head)
62     if kind in {"exe", "zip", "binary", "unknown"}:
63         return jsonify({"error": "Unsafe or unsupported file content"}), 400
64
65     if kind not in EXTENSION_TO_KIND[ext]:
66         return jsonify({"error": "File content does not match extension"}), 400
67
68     file.stream.seek(0)
69     stored_name = f"{uuid.uuid4().hex}_{original_name}"
70     destination = UPLOAD_DIR / stored_name
71     file.save(destination)
72
73     return jsonify({"message": "Upload successful", "file": stored_name}), 201
74
75 @app.errorhandler(413)
76 def file_too_large():
77     return jsonify({"error": "File too large. Max size is 5 MB"}), 413
78
79 if __name__ == "__main__":
80     app.run(debug=False)
```

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

```

1 import re
2 import subprocess
3
4 ALLOWED_COMMANDS = [
5     "ping": ["window", "-n", "-w"], "time": ["-n", "-w"],
6     "nslookup": ["window", "-n", "-w"], "time": [{"-n", "-w"}],
7 ]
8 SAFE_TARGET_PATTERN = re.compile(r"^(?!(?:0|1|255|256))$")
9
10 def validate_target(target: str) -> str:
11     value = target.strip()
12     if not SAFE_TARGET_PATTERN.fullmatch(value):
13         raise ValueError("Invalid target")
14     if value.startswith(".") or value.endswith(".") or "." in value:
15         raise ValueError("Invalid target")
16     return value
17
18 def validate_count(count: str) -> int:
19     value = int(count)
20     if value < 1 or value > 10:
21         raise ValueError("Count must be between 1 and 10")
22     return value
23
24 def run_safe(command: str, target: str, count: int = 4) -> str:
25     cmd = command.strip().lower()
26     if cmd not in ALLOWED_COMMANDS:
27         raise ValueError("Unsupported command")
28     safe_target = validate_target(target)
29     safe_count = validate_count(str(count))
30     if cmd == "ping":
31         args = ["ping", "-n", str(safe_count), safe_target]
32     else:
33         args = ["nslookup", safe_target]
34     result = subprocess.run(args, capture_output=True, text=True, timeout=15, shell=False, check=False)
35     output = result.stdout if result.stdout else result.stderr
36     return output.strip()
37
38 if __name__ == "__main__":
39     user_command = input("Command (ping/nslookup): ")
40     user_target = input("Target (domain or host): ")
41     user_count = input("Ping count (1-10, default 4): ").strip() or "4"
42     try:
43         print(run_safe(user_command, user_target, int(user_count)))
44     except Exception as e:
45         print(f"Error: {e}")

```

```

C:\Users\kruth > AAC A 20.5.py > run_safe
24 def run_safe(command: str, target: str, count: int = 4) -> str:
25     cmd = command.strip().lower()
26     if cmd not in ALLOWED_COMMANDS:
27         raise ValueError("Unsupported command")
28     safe_target = validate_target(target)
29     safe_count = validate_count(str(count))
30     if cmd == "ping":
31         args = ["ping", "-n", str(safe_count), safe_target]
32     else:
33         args = ["nslookup", safe_target]
34     result = subprocess.run(args, capture_output=True, text=True, timeout=15, shell=False, check=False)
35     output = result.stdout if result.stdout else result.stderr
36     return output.strip()
37
38 if __name__ == "__main__":
39     user_command = input("Command (ping/nslookup): ")
40     user_target = input("Target (domain or host): ")
41     user_count = input("Ping count (1-10, default 4): ").strip() or "4"
42     try:
43         print(run_safe(user_command, user_target, int(user_count)))
44     except Exception as e:
45         print(f"Error: {e}")

```

Task 5 – Secure Input Handling in Web Forms

Task:

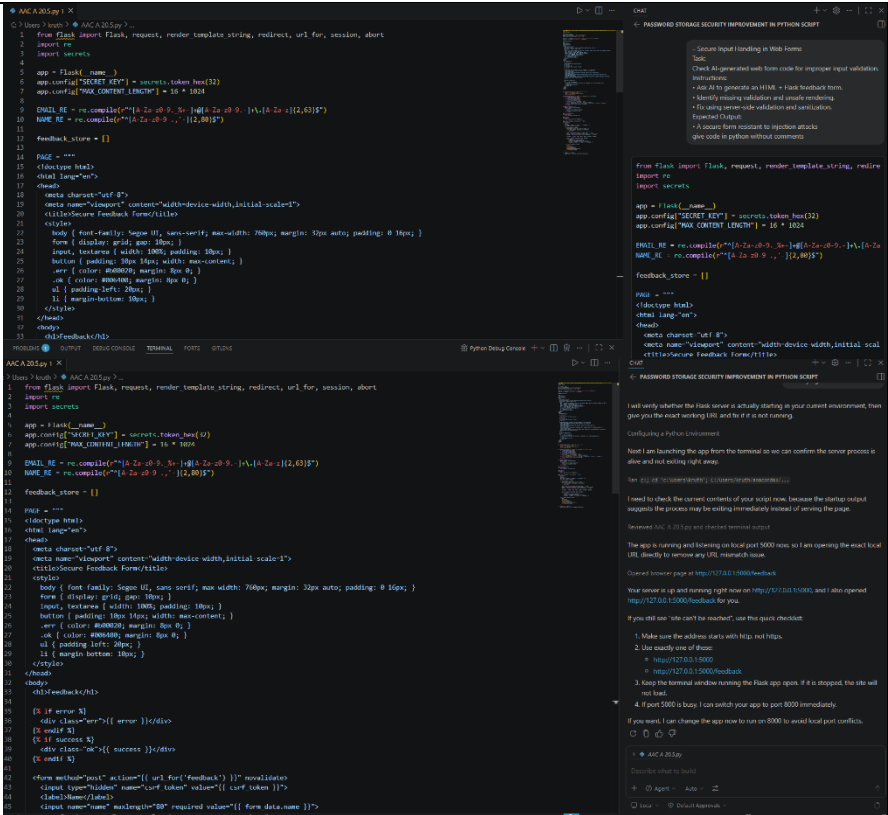
Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.



← → ↻ http://127.0.0.1:5000/feedback

Feedback

Feedback submitted successfully.

Name

Email

Message

Submit

Recent Feedback

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.